

RAPPORT TECHNIQUE D'INGÉNIERIE LOGICIELLE

REFACTORISATION D'ARCHITECTURE & MODERNISATION DU CODE

MIGRATION ARCHITECTURALE

Du Paradigme Procédural vers la Programmation
Orientée Objet (POO & MVC)

Étude de Cas Approfondie, Conception des Modèles DAO, Patron Singleton,
Routage Dynamique et Gestion Atomique des Notes Scolaires

Projet NotesEcole

Auteur :

Équipe de Développement

Département Ingénierie Web

Technologies Clés :

PHP 8.2+ | PostgreSQL 16

MVC | POO | PDO | Git

Session Académique & Professionnelle – Février 2026

Dépôt de Code : NotesEcole (Branches main & P00)

Résumé Exécutif

Le présent document constitue un rapport d'ingénierie logicielle exhaustif retraçant l'ensemble du cycle de refactorisation mené sur la plateforme web de gestion scolaire **NotesEcole**. Initialement développé selon une approche exclusivement **procédurale** sur la branche principale (**main**), le projet présentait des limitations structurelles inhérentes à ce paradigme : prolifération de fonctions globales, couplage fort avec l'instance de connexion à la base de données PostgreSQL via l'injection manuelle répétitive de la variable `$pdo`, dispersion de la logique métier et vulnérabilités organisationnelles face à la montée en charge.

Ce mémoire documente la transition intégrale du système vers le paradigme de la **Programmation Orientée Objet (POO)** articulé autour de l'architecture **Modèle-Vue-Contrôleur (MVC)** sur la branche dédiée **P00**. Nous analysons les fondements théoriques de cette refonte (principes SOLID, découplage, cohésion forte, encapsulation), la mise en place de patrons de conception éprouvés (*Singleton* pour le gestionnaire de base de données, *Data Access Object / Repository* pour les 10 entités scolaires, *Front Controller* et *Router* dynamique orienté verbes HTTP), ainsi que la sécurisation transactionnelle des enregistrements de notes scolaires en masse (ACID).

Mots-clés : Programmation Orientée Objet, Refactorisation, PHP 8, PostgreSQL, Patron MVC, Singleton, DAO, Routage HTTP, Transactions ACID, Notes Scolaires.

Executive Abstract

This technical engineering report provides an in-depth account of the architectural refactoring performed on the **NotesEcole** school grading management platform. Initially conceived following a purely **procedural** paradigm on the **main** branch, the system suffered from systemic technical debt: global scope namespace pollution, severe coupling caused by passing the PDO database handle across every functional layer, lack of state encapsulation, and poor testability.

This document details the step-by-step migration to a modern **Object-Oriented Programming (OOP)** architecture grounded in the **Model-View-Controller (MVC)** design pattern on the **P00** branch. We demonstrate how software engineering principles (SOLID, high cohesion, low coupling) guided the implementation of key architectural design patterns: a thread-safe PDO *Singleton* with transaction management, an extensible HTTP *Router* and *Front Controller*, static Session management with transient flash messages, and 10 dedicated *Data Access Object (DAO)* models mirroring the PostgreSQL relational schema. Furthermore, performance optimizations and atomic multi-row grade submissions are formally verified.

Keywords: Object-Oriented Programming, Refactoring, PHP 8, PostgreSQL, MVC Pattern, Singleton, DAO, Dynamic Routing, ACID Transactions, Academic Grading System.

Avant-propos & Remerciements

Ce travail de refactorisation logicielle s'inscrit dans une volonté continue d'excellence technique, de conformité aux standards contemporains du développement Web (PHP FIG, PSR, programmation défensive) et de pérennisation des applications de gestion des données critiques.

Nous tenons à exprimer notre profonde gratitude envers l'ensemble des encadrants techniques, des relecteurs de code et des parties prenantes pédagogiques ayant contribué à la définition rigoureuse des règles de calcul des moyennes scolaires pondérées et des contraintes d'intégrité de la base de données.

Table des matières

Résumé Exécutif	i
Executive Abstract	ii
Avant-propos & Remerciements	iii
1 Introduction & Contexte du Projet	1
1.1 Mise en Contexte et Présentation du Projet NotesEcole	1
1.2 Le Périmètre Fonctionnel et les Règles Métier Scolaires	2
1.3 Problématique : Les Limites de l'Approche Procédurale Initiale	2
1.4 Objectifs de la Refactorisation vers la POO	3
2 Fondements Théoriques & Comparaison des Paradigmes	5
2.1 Le Paradigme Procédural : Principes, Avantages et Limites	5
2.1.1 Caractéristiques Principales du Code Procédural	5
2.1.2 Limitations Critiques Constatées dans un Projet Évolutif	5
2.2 Le Paradigme Orienté Objet (POO) : Les 4 Piliers Fondamentaux	6
2.2.1 Application des Piliers au Projet NotesEcole	6
2.3 Les Principes SOLID et leur Implémentation	7
2.3.1 Concrétisation des Principes dans NotesEcole	7
2.4 Tableau Synthétique Comparatif : Procédural vs POO	7
3 Modélisation des Données & Schéma SQL PostgreSQL	9
3.1 Modèle Conceptuel et Logique des Données (MCD / MLD)	9
3.2 Description des Tables et Contraintes d'Intégrité	10
3.2.1 Dictionnaire des Tables Principales	10
3.3 Analyse des Requêtes SQL Complexes et Calculs d'Agrégation	11
3.3.1 1. Extraction de la Grille de Saisie des Notes	11
3.3.2 2. Calcul de la Moyenne de Classe par Matière (avec exclusion des notes nulles)	11
3.3.3 3. Calcul de la Moyenne Générale Globale de la Classe (Multi-Matières)	12
4 Autopsie de l'Architecture Procédurale Historique	13
4.1 Organisation des Fichiers et Flux d'Exécution sur la Branche <code>main</code>	13
4.2 Analyse Détaillée des Composants Procéduraux Initiaux	14

4.2.1	1. Le Gestionnaire de Base de Données Procédural (<code>Database.php</code>)	14
4.2.2	2. Le Système de Routage Procédural (<code>Router.php</code>)	14
4.2.3	3. Le Contrôleur et la Logique de Saisie Procéduraux	15
4.3	Bilan de la Dette Technique Procédurale	16
5	Démarche Méthodologique & Stratégie de Migration	17
5.1	Principes Directeurs de la Refactorisation	17
5.2	Détail des Étapes de Transition	18
5.2.1	Étape 1 : Sécurisation et Unification du Noyau (<code>app/core/</code>)	18
5.2.2	Étape 2 : Éclatement et Spécialisation des Modèles (<code>app/models/</code>)	18
5.2.3	Étape 3 : Refactorisation des Contrôleurs (<code>app/controllers/</code>)	18
5.2.4	Étape 4 : Modernisation du Point d'Entrée Unique (<code>public/index.php</code>)	18
5.3	Stratégie de Gestion de Version avec Git	19
6	Conception de la Nouvelle Architecture POO & MVC	20
6.1	Le Patron MVC Appliqué à NotesEcole	20
6.2	Le Noyau Applicatif (<code>app/core/</code>)	21
6.2.1	1. Le Patron Singleton pour la Classe <code>Database</code>	21
6.2.2	2. Le Routeur Dynamique Orienté Verbes HTTP (<code>Router.php</code>)	22
6.2.3	3. Gestionnaire de Session & Notifications Flash (<code>Session.php</code>)	23
7	Étude Comparative Détaillée des Composants Métier	25
7.1	La Couche Modèle : Spécialisation et Encapsulation	25
7.1.1	1. Le Modèle d'Évaluation (<code>EvaluationModel.php</code>)	25
7.1.2	2. Le Modèle d'Authentification (<code>UtilisateurModel.php</code>)	26
7.2	La Couche Contrôleur : Orchestration et Injection de Dépendances	27
7.2.1	1. Le Contrôleur de Notes (<code>NoteController.php</code>)	27
7.2.2	2. Sauvegarde par Lot et Robustesse de l'Action <code>save()</code>	27
8	Sécurité, Transactions ACID & Qualité du Code	29
8.1	Garantie d'Intégrité par les Transactions ACID	29
8.1.1	Le Problème de l'Écriture Partielle	29
8.1.2	Implémentation Transactionnelle dans <code>saveNotes()</code>	29
8.2	Sécurisation contre les Injections SQL via Requêtes Préparées	31
8.3	Typage Strict et Modernité PHP 8.x	31
9	Guide de Déploiement & Perspectives d'Évolution	32
9.1	Guide de Déploiement et Configuration de l'Application	32
9.1.1	Prérequis Système	32
9.1.2	Initialisation de la Base de Données	32
9.1.3	Lancement de l'Application en Environnement Local	33
9.2	Guide de Compilation du Document LaTeX	33
9.2.1	Compilation en Ligne de Commande	33
9.3	Perspectives d'Évolution et Feuille de Route (*Roadmap*)	33

10 Conclusion Générale & Bilan Technique	35
10.1 Synthèse de la Transformation Logicielle	35
10.2 Bilan Quantitatif et Qualitatif	35
10.2.1 Indicateurs Qualitatifs	35
10.2.2 Indicateurs Métriques du Dépôt Git	36
A Dictionnaire de Données Exhaustif	37
A.1 Détail des Tables du Schéma PostgreSQL	37
B Diagramme de Classes UML Global	39
C Glossaire des Termes d'Ingénierie Logicielle	40
D Références & Bibliographie	41

Table des figures

1.1	Schéma conceptuel des flux et acteurs du système NotesEcole	2
2.1	Les quatre piliers conceptuels de la Programmation Orientée Objet	6
3.1	Diagramme Entité-Association simplifié du schéma relationnel	10
4.1	Flux d'exécution procédural et dépendances sur la branche <code>main</code>	13
5.1	Chronologie séquentielle des phases de refactorisation	17
6.1	Flux d'interactions de l'architecture MVC cible de NotesEcole	20
8.1	Mécanisme d'immunisation contre les injections SQL par séparation du code et des données	31
9.1	Feuille de route technique pour les versions futures de NotesEcole	33
10.1	Les quatre piliers de la valeur ajoutée apportée par la refonte POO	35
B.1	Diagramme de classes UML global de l'architecture POO & MVC	39

Liste des tableaux

2.1	Comparaison multidimensionnelle entre l'approche procédurale et l'architecture POO	8
A.1	Dictionnaire exhaustif des données du projet NotesEcole	38

Listings

3.1	Requête d'extraction matricielle des notes avec jointure externe	11
3.2	Calcul statistique de la moyenne de classe par matière	11
3.3	Calcul de la moyenne globale de la classe avec expansion de tableau PostgreSQL	12
4.1	Module de base de données procédural historique	14
4.2	Système de routage procédural sur la branche main	14
4.3	Contrôleur d'authentification procédural	15
5.1	Front Controller déclaratif en POO	18
6.1	Implémentation complète du Singleton Database	21
6.2	La classe Router et son algorithme de dispatching	22
6.3	Gestion des sessions statiques et messages Flash	23
7.1	Structure et méthodes de la classe EvaluationModel	25
7.2	Vérification des identifiants au sein d'UtilisateurModel	26
7.3	Constructeur et injection de dépendances dans NoteController	27
7.4	Traitement de sauvegarde groupée et redirection PRG	27
8.1	Implémentation de l'atomicité transactionnelle dans EvaluationModel	30
9.1	Commandes d'initialisation de la base PostgreSQL	32
9.2	Lancement du serveur PHP natif	33
9.3	Compilation de la documentation via pdflatex	33

Introduction & Contexte du Projet

1.1 Mise en Contexte et Présentation du Projet NotesEcole

Dans le contexte de la transformation numérique des établissements scolaires, la gestion rigoureuse, transparente et automatisée des évaluations pédagogiques représente un enjeu capital pour le corps professoral et les directions d'études. Le projet **NotesEcole** est une application web conçue pour centraliser, administrer et automatiser la saisie des évaluations périodiques ainsi que le calcul instantané des moyennes pondérées des élèves.

L'application s'adresse à différents profils d'utilisateurs institutionnels :

1. **La Direction de l'Établissement** : supervise l'ensemble des structures académiques, valide les années scolaires actives, consulte les statistiques globales de performance des classes et assure la gouvernance pédagogique.
2. **Le Corps Enseignant (Professeurs)** : renseigne périodiquement les notes des élèves inscrits dans leurs classes respectives pour leurs matières d'attribution (Devoir 1, Devoir 2, Composition de fin de trimestre).
3. **Les Surveillants et Personnels Administratifs** : gèrent les inscriptions annuelles des élèves, l'affectation aux cohortes de classes et l'intégrité des listes d'élèves.

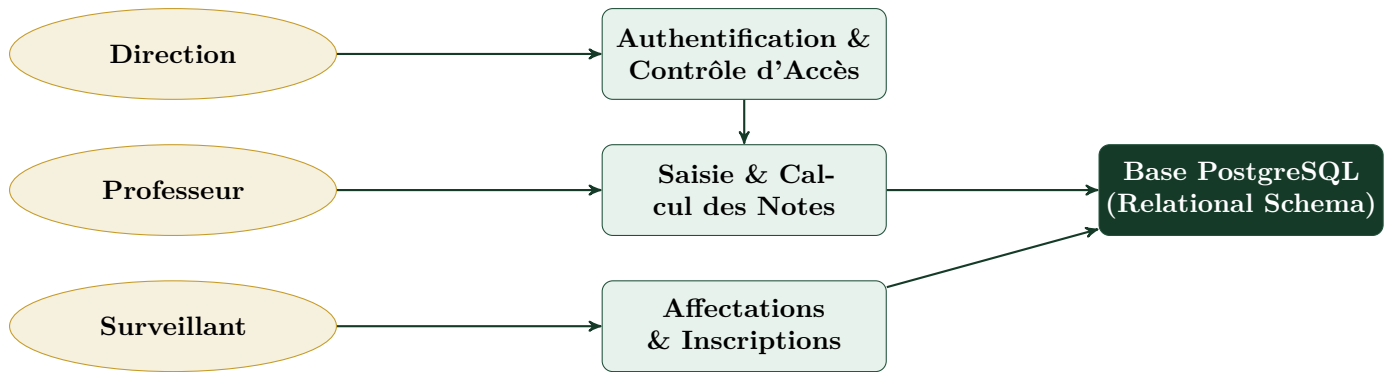


FIGURE 1.1 – Schéma conceptuel des flux et acteurs du système NotesEcole

1.2 Le Périmètre Fonctionnel et les Règles Métier Scolaires

Le système repose sur un ensemble précis de règles de gestion et d'exigences pédagogiques formalisées :

- **Structure Temporelle et Année Active :** Le système opère sur une année scolaire active unique à un instant donné (ex. 2025-2026), subdivisée en périodes d'évaluation normalisées (Trimestre 1, Trimestre 2, Trimestre 3).
- **Inscriptions et Cohortes :** Un élève est rattaché à une classe (ex. 3ème A, 4ème B) pour une année scolaire donnée via une table de jointure explicite `inscriptions`.
- **Curriculum par Classe :** Chaque classe dispose d'un panier de matières obligatoires (Mathématiques, Français, Histoire-Géographie, Sciences Physiques) associées via la table `matiere_classes`.
- **Formule Pédagogique de Calcul de la Moyenne par Matière :** La note finale trimestrielle dans une matière M pour un élève E obéit à la règle de pondération standard où la composition semestrielle/trimestrielle possède un coefficient double par rapport aux devoirs continus :

$$\text{Moyenne}_{\text{matière}} = \frac{\text{Devoir}_1 + \text{Devoir}_2 + (2 \times \text{Composition})}{4} \quad (1.1)$$

- **Gestion des Moyennes Globales et Exclusion des Vides :** Lors du calcul de la moyenne générale d'une classe ou d'une matière, les élèves n'ayant composé à aucune épreuve ou présentant une moyenne calculée strictement égale à zéro sont filtrés afin de ne pas fausser statistiquement les performances réelles de la classe.

1.3 Problématique : Les Limites de l'Approche Procédurale Initiale

À sa création sur la branche `main`, le projet a été implémenté en utilisant un style procédural traditionnel en PHP natif. Bien que cette approche ait permis d'obtenir rapidement un premier

prototype fonctionnel (MVP), l'accroissement de la complexité a immédiatement mis en lumière des faiblesses architecturales majeures :

1. **Couplage Excessif et Fardeau du Handle de Base de Données** : Chaque fonction de modèle (ex. `get_eleves_notes($pdo, ...)`, `sauvegarder_notes($pdo, ...)`, `verify_user_credentials($pdo, ...)`) nécessitait la transmission explicite et redondante de l'objet `$pdo`.
2. **Pollution du Scope Global et Risque de Conflit de Noms** : Les fonctions d'authentification (`login()`, `logout()`) et de gestion des notes (`indexNote()`, `enregistrerNote()`) résidaient dans l'espace de nommage global, empêchant toute encapsulation des états intermédiaires.
3. **Gestion de Session Désarticulée** : L'absence de structure unifiée pour les sessions entraînait des appels épars à `session_start()` et des vérifications manuelles directes du tableau superglobal `$_SESSION`.
4. **Absence de Polymorphisme et Extensibilité Faible** : L'ajout d'une nouvelle entité nécessitait l'écriture d'une multitude de scripts procéduraux indépendants sans contrat d'interface ni uniformité dans la gestion des erreurs.

Constat Technique Initial

Le code procédural initial, bien que modulaire en apparence via des découpages de fichiers, souffrait d'une forte dette technique. L'état de l'application n'était pas maîtrisé, et les dépendances n'étaient ni isolées ni injectées proprement, rendant les tests unitaires et la maintenance évolutive quasi impossibles.

1.4 Objectifs de la Refactorisation vers la POO

La refonte complète menée sur la branche P00 s'est assignée des objectifs rigoureux de qualité logicielle :

- **Encapsulation Stricte** : Regrouper la logique métier et l'accès aux données au sein d'objets cohérents dissimulant leurs détails d'implémentation internes.
- **Adoption des Design Patterns Éprouvés** :
 - *Singleton Pattern* pour la classe `Database`, garantissant l'unicité de la connexion PDO et la gestion native des transactions.
 - *DAO / Repository Pattern* pour les 10 modèles scolaires, instanciant ou exploitant la connexion sans passage explicite de paramètres.
 - *Front Controller & Dynamic Router* pour le dispatching des requêtes basé sur les verbes HTTP (GET / POST).
- **Typage Strict et Sécurité PHP 8+** : Typage fort des propriétés d'instance, des paramètres et des valeurs de retour (`int`, `float`, `array`, `?Database`, `void`).

- **Sécurisation Transactionnelle des Écritures (ACID) :** Garantie d'atomicité totale lors de la sauvegarde simultanée des notes d'une classe complète.

Fondements Théoriques & Comparaison des Paradigmes

2.1 Le Paradigme Procédural : Principes, Avantages et Limites

Le paradigme procédural repose sur le concept d'appels de procédures ou de fonctions exécutant séquentiellement une suite d'instructions algorithmiques. Dans ce modèle, les structures de données (généralement des tableaux associatifs ou des variables scalaires en PHP natif) sont distinctes et découplées des fonctions chargées de les manipuler.

2.1.1 Caractéristiques Principales du Code Procédural

- **Flux Linéaire et Séquentiel** : L'exécution progresse pas à pas au travers d'un enchaînement de fonctions appelées de manière hiérarchique ou séquentielle.
- **Données Passives** : Les données manipulées n'ont aucune conscience de leur propre état ni des invariants métier qui leur sont applicables. Une note d'évaluation transmise sous forme de tableau ['devoir1' => 15, 'devoir2' => 12] ne possède aucune méthode intrinsèque de validation ou de calcul de moyenne.
- **Passage Explicite des Ressources** : Les ressources partagées (telles que le descripteur de connexion à la base de données `$pdo`) doivent être transmises en argument à chaque appel fonctionnel.

2.1.2 Limitations Critiques Constatées dans un Projet Évolutif

Bien que le paradigme procédural offre une courbe d'apprentissage directe et une rapidité initiale pour l'écriture de scripts simples, il devient contre-productif dès lors que la base de code grandit :

1. **Explosion du Couplage** : Toute modification de la signature d'une fonction de base (par exemple l'ajout d'un paramètre de configuration à la connexion DB) exige de modifier l'intégralité des fonctions appelantes dans toute l'arborescence.
2. **Effets de Bord Incontrôlés (*Side-Effects*)** : Les variables globales ou les modifications indirectes de tableaux par référence rendent difficile la prévisibilité de l'état du système à un instant t .
3. **Absence de Protection de l'État** : Il est impossible d'empêcher un script tiers de corrompre directement un tableau de données, aucune visibilité (`private`, `protected`) n'existant pour garantir l'intégrité de l'état interne.

2.2 Le Paradigme Orienté Objet (POO) : Les 4 Piliers Fondamentaux

La Programmation Orientée Objet structure les programmes sous la forme d'un ensemble d'**objets** autonomes et communicants, combinant au sein d'une même entité des **données** (attributs / propriétés) et des **traitements** (méthodes).

Les 4 Piliers de la Programmation Orientée Objet

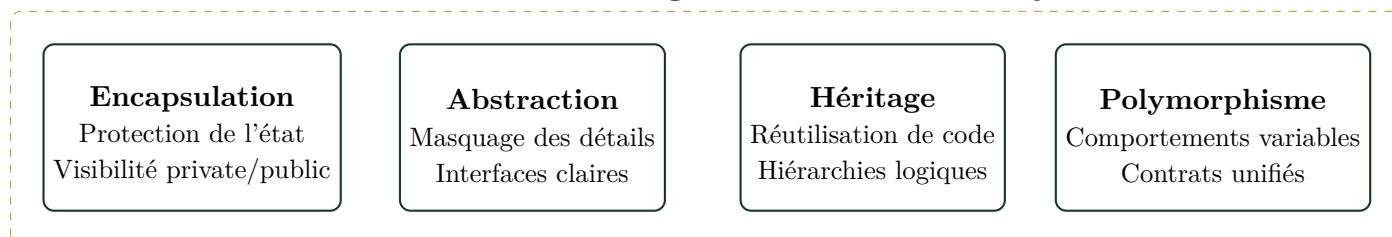


FIGURE 2.1 – Les quatre piliers conceptuels de la Programmation Orientée Objet

2.2.1 Application des Piliers au Projet NotesEcole

- **Encapsulation** : La classe `Database` encapsule l'objet interne PDO `$pdo` et le conserve en `private`. Aucun contrôleur ni modèle n'a accès direct au driver brut, passant exclusivement par des méthodes normalisées (`executeQuery`, `executeUpdate`, `beginTransaction`).
- **Abstraction** : La classe `EvaluationModel` offre des méthodes de haut niveau telles que `saveNotes()` ou `getMoyenneGeneral()`, masquant la complexité des requêtes SQL de jointure multi-tables et des calculs matriciels PostgreSQL.
- **Modularité** : Chaque domaine métier (élèves, classes, matières, évaluations) est représenté par une classe dédiée responsable de son propre cycle de vie et de son accès aux données.

2.3 Les Principes SOLID et leur Implémentation

Les principes SOLID constituent le standard de l'ingénierie logicielle pour concevoir des architectures robustes, maintenables et évolutives.

Les 5 Principes SOLID

- S – Single Responsibility Principle (SRP) :** Une classe ne doit avoir qu'une seule et unique raison de changer.
- O – Open/Closed Principle (OCP) :** Les entités logicielles doivent être ouvertes à l'extension mais fermées à la modification.
- L – Liskov Substitution Principle (LSP) :** Les objets d'un type dérivé doivent pouvoir remplacer les objets du type de base sans altérer la cohérence du programme.
- I – Interface Segregation Principle (ISP) :** Aucun client ne doit être contraint de dépendre de méthodes qu'il n'utilise pas.
- D – Dependency Inversion Principle (DIP) :** Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau, mais d'abstractions.

2.3.1 Concrétisation des Principes dans NotesEcole

Dans le cadre de la refactorisation de NotesEcole :

- **SRP Respecté :** Dans la version procédurale, `NoteController.php` gérait simultanément l'affichage, les calculs de moyennes, les requêtes SQL et la manipulation brute des sessions. Dans la version POO, les responsabilités sont rigoureusement cloisonnées :
 - `Router` est exclusivement responsable de l'analyse d'URI et du dispatching d'actions.
 - `Session` centralise l'état volatile et les messages temporaires (*flash messages*).
 - `EvaluationModel` encapsule la persistance et les requêtes SQL d'évaluation.
 - `NoteController` se limite à l'orchestration des flux et à l'aiguillage vers les vues.
- **DIP & Injection par Constructeur :** Les contrôleurs (`NoteController`, `AuthController`) initialisent leurs dépendances au sein de leur constructeur `__construct()`, instanciant leurs modèles métier de façon structurée et typée.

2.4 Tableau Synthétique Comparatif : Procédural vs POO

Le tableau 2.1 récapitule les divergences fondamentales entre l'implémentation initiale et l'architecture refactorisée.

Critère d'Évaluation	Approche Procédurale (Branche main)	Approche POO & MVC (Branche P00)
Organisation du Code	Fonctions éparpillées dans des fichiers inclus via <code>require_once</code> .	Classes structurées en couches logiques (<code>core</code> , <code>controllers</code> , <code>models</code> , <code>views</code>).
Gestion de la Connexion DB	Variable <code>\$pdo</code> transmise manuellement en argument à chaque fonction.	Classe <code>Database</code> en <i>Singleton</i> , accessible via <code>Database::getInstance()</code> .
Encapsulation & État	Inexistante : tableaux associatifs bruts modifiables depuis n'importe quel point.	Élevée : propriétés privées, méthodes d'accès, typage strict des données.
Routage HTTP	Boucle itérative sur un tableau statique de chaînes sans distinction GET/POST.	Classe <code>Router</code> avec enregistrement dynamique selon les verbes HTTP (<code>get()</code> , <code>post()</code>).
Gestion des Sessions	Fonctions globales avec accès direct à <code>\$_SESSION</code> et risque d'écrasement.	Classe <code>Session</code> avec méthodes statiques et système de messages flash transitoires.
Gestion des Transactions	Appels directs à <code>\$pdo->beginTransaction()</code> dispersés dans les modèles.	Méthodes d'encapsulation transactionnelle sécurisées (<code>beginTransaction</code> , <code>commit</code> , <code>rollBack</code>).
Évolutivité & Tests	Difficile : adhérence forte au runtime global et aux superglobales.	Excellente : isolation unitaire possible, mock des modèles et contrôleurs facilité.

TABLE 2.1 – Comparaison multidimensionnelle entre l'approche procédurale et l'architecture POO

Modélisation des Données & Schéma SQL PostgreSQL

3.1 Modèle Conceptuel et Logique des Données (MCD / MLD)

Le système **NotesEcole** s'appuie sur une base de données relationnelle propulsée par le SGBD **PostgreSQL** (version 16). La modélisation a été rigoureusement conçue afin de respecter la 3^{ème} Forme Normale (3FN), évitant toute redondance informationnelle et garantissant une intégrité référentielle stricte.

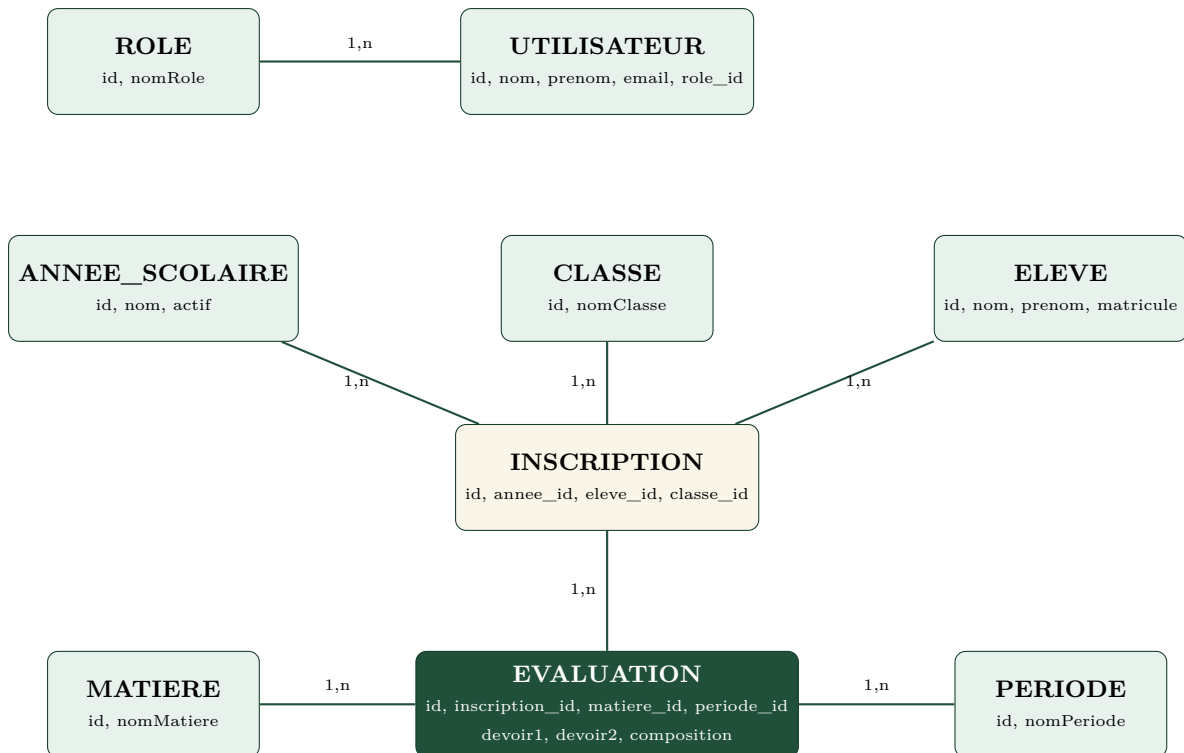


FIGURE 3.1 – Diagramme Entité-Association simplifié du schéma relationnel

3.2 Description des Tables et Contraintes d'Intégrité

Le schéma comprend 10 tables interconnectées par des clés étrangères avec propagation de suppression (ON DELETE CASCADE) pour préserver la consistance des données lors de purges ou de réinitialisations académiques.

3.2.1 Dictionnaire des Tables Principales

1. **anneesScolaires** : Définit les années académiques (ex. '2025-2026'). La colonne **actif** INT identifie l'année active par défaut (1 pour active, 0 sinon).
2. **eleves** : Enregistre l'identité civile de l'apprenant (**nom**, **prenom**) ainsi qu'un identifiant institutionnel unique (**matricule** UNIQUE).
3. **classes** : Stocke les dénominations des cohortes scolaires (**nomClasse** UNIQUE, ex. '3em A', '4em B').
4. **inscriptions** : Table pivot matérialisant l'inscription d'un élève dans une classe pour une année scolaire déterminée.
5. **roles & utilisateurs** : Système d'authentification RBAC (*Role-Based Access Control*) intégrant le hachage sécurisé des mots de passe.
6. **matieres & matiere_classes** : Catalogue des disciplines scolaires et association des matières autorisées par niveau de classe.
7. **periodes** : Découpage temporel de l'évaluation (ex. 'Trimestre 1').

8. **evaluations** : Stocke les notes obtenues par inscription pour une matière et une période (devoir1 NUMERIC(4,2), devoir2 NUMERIC(4,2), composition NUMERIC(4,2)).

3.3 Analyse des Requêtes SQL Complexes et Calculs d'Agrégation

La pertinence d'un système de gestion de notes repose sur sa capacité à restituer fidèlement des grilles de saisie exhaustives et à agréger les moyennes selon des règles mathématiques strictes.

3.3.1 1. Extraction de la Grille de Saisie des Notes

Lorsqu'un enseignant sélectionne une classe, une matière et une période, le système doit afficher la totalité des élèves inscrits, qu'ils possèdent déjà une note ou non. Une jointure externe gauche (LEFT JOIN) est donc indispensable.

```

1  SELECT
2      i.id AS inscription_id,
3      e.id AS eleve_id,
4      e.nom,
5      e.prenom,
6      e.matricule,
7      ev.id AS evaluation_id,
8      COALESCE(ev.devoir1, 0) AS devoir1,
9      COALESCE(ev.devoir2, 0) AS devoir2,
10     COALESCE(ev.composition, 0) AS composition
11 FROM eleves e
12 JOIN inscriptions i ON i.eleve_id = e.id
13     AND i.annee_id = :annee_id
14     AND i.classe_id = :classe_id
15 LEFT JOIN evaluations ev ON ev.inscription_id = i.id
16     AND ev.matiere_id = :matiere_id
17     AND ev.periode_id = :periode_id
18 ORDER BY e.nom ASC, e.prenom ASC;
```

Listing 3.1 – Requête d'extraction matricielle des notes avec jointure externe

3.3.2 2. Calcul de la Moyenne de Classe par Matière (avec exclusion des notes nulles)

Le calcul de la moyenne générale d'une classe pour une discipline donnée applique la pondération officielle et exclut les élèves n'ayant pas composé (`moyenne_eleve > 0`) :

```

1  SELECT ROUND(COALESCE(AVG(moyenne_eleve), 0), 2) AS moyenne
2  FROM (
3      SELECT
4          i.id AS inscription_id,
5          ROUND((COALESCE(ev.devoir1, 0) + COALESCE(ev.devoir2, 0) + 2 *
6              COALESCE(ev.composition, 0)) / 4.0, 2) AS moyenne_eleve
```

```

6      FROM inscriptions i
7      LEFT JOIN evaluations ev
8          ON ev.inscription_id = i.id
9          AND ev.matiere_id = :matiere_id
10         AND ev.periode_id = :periode_id
11      WHERE i.annee_id = :annee_id
12         AND i.classe_id = :classe_id
13 ) AS moyennes_eleves
14 WHERE moyenne_eleve > 0;

```

Listing 3.2 – Calcul statistique de la moyenne de classe par matière

3.3.3 3. Calcul de la Moyenne Générale Globale de la Classe (Multi-Matières)

Pour déterminer la moyenne globale de l'ensemble de la classe sur toutes les matières de son cursus, la requête exploite les fonctionnalités avancées de PostgreSQL (`unnest(ARRAY[...])`) afin de générer le produit cartésien des matières attribuées, puis calcule la moyenne des moyennes individuelles :

```

1  SELECT ROUND(COALESCE(AVG(moyenne_eleve), 0), 2) AS moyenne
2  FROM (
3      SELECT
4          i.id AS inscription_id,
5          ROUND(COALESCE(SUM(ROUND((COALESCE(ev.devoir1, 0) +
6              COALESCE(ev.devoir2, 0) + 2 * COALESCE(ev.composition, 0)) / 4.0,
7              2)), 0) / :nb_matiere, 2) AS moyenne_eleve
8      FROM inscriptions i
9      CROSS JOIN (SELECT unnest(ARRAY[1, 2, 3, 4]) AS matiere_id) m
10     LEFT JOIN evaluations ev
11         ON ev.inscription_id = i.id
12         AND ev.matiere_id = m.matiere_id
13         AND ev.periode_id = :periode_id
14     WHERE i.annee_id = :annee_id
15         AND i.classe_id = :classe_id
16     GROUP BY i.id
17 ) AS moyennes_eleves
18 WHERE moyenne_eleve > 0;

```

Listing 3.3 – Calcul de la moyenne globale de la classe avec expansion de tableau PostgreSQL

Autopsie de l'Architecture Procédurale Historique

4.1 Organisation des Fichiers et Flux d'Exécution sur la Branche main

Avant la refonte, le projet **NotesEcole** sur la branche `main` s'articulait autour d'un ensemble de scripts procéduraux divisés en répertoires `app/core`, `app/models`, `app/controllers` et `app/views`. Bien qu'une intention de découpage en couches fût présente, l'absence de structures de classes imposait un fonctionnement basé sur des variables globales et des inclusions directes de scripts.

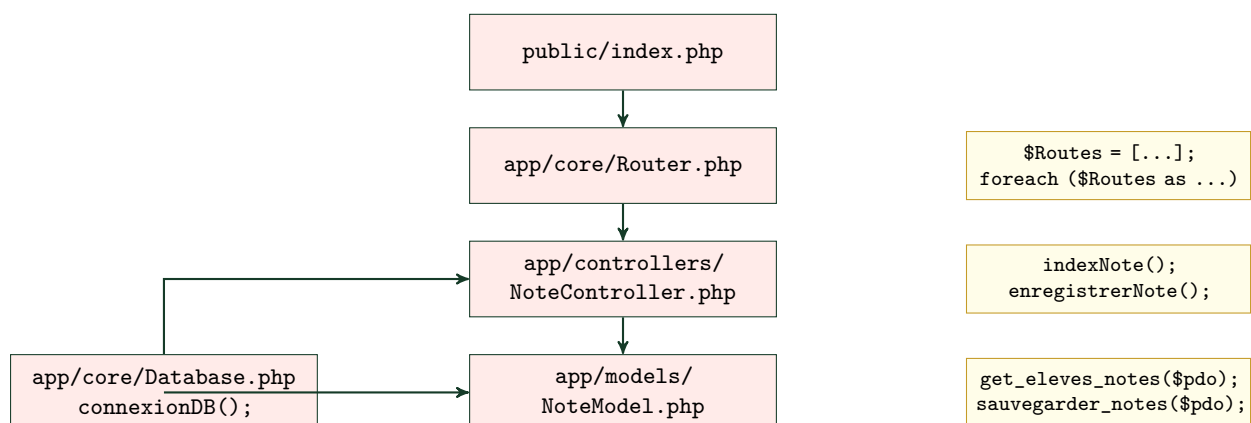


FIGURE 4.1 – Flux d'exécution procédural et dépendances sur la branche `main`

4.2 Analyse Détaillée des Composants Procéduraux Initiaux

4.2.1 1. Le Gestionnaire de Base de Données Procédural (Database.php)

Dans la version initiale, la connexion PDO était générée par une fonction `connexionDB()` encapsulant une variable statique locale. Cependant, chaque fonction utilitaire d'exécution de requête imposait le passage explicite de cette ressource PDO en premier paramètre :

```
1 <?php
2 // app/core/Database.php (Branche main)
3
4 function connexionDB(): PDO {
5     static $pdo = null;
6     if ($pdo === null) {
7         try {
8             $pdo = new
9                 PDO("pgsql:host=localhost;dbname=notes_ecole;port=5433",
10                    "ichigo", "password");
11             $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
12                               PDO::FETCH_ASSOC);
13             $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
14         } catch (Exception $ex) {
15             die('Erreur : ' . $ex->getMessage());
16         }
17     }
18     return $pdo;
19 }
20
21 function executeQuery(PDO $pdo, string $sql, array $datas, bool $single =
22     false): array {
23     $statement = $pdo->prepare($sql);
24     $statement->execute($datas);
25     $result = $single ? $statement->fetch() : $statement->fetchAll();
26     return $result !== false ? $result : [];
```

Listing 4.1 – Module de base de données procédural historique

4.2.2 2. Le Système de Routage Procédural (Router.php)

Le routage était réalisé par une itération linéaire sur une matrice de routes sous forme de tableau. Aucune distinction n'était faite entre les verbes HTTP (GET, POST, etc.), ouvrant la porte à des failles potentielles lors de soumissions de formulaires :

```
1 <?php
2 // app/core/Router.php (Branche main)
3
4 $Routes = [
5     ['/', 'AuthController.php', 'login'],
6     ['/login', 'AuthController.php', 'login'],
7     ['/logout', 'AuthController.php', 'logout'],
8     ['/gestion', 'NoteController.php', 'indexNote'],
```



```

9      ['/gestion/save', 'NoteController.php', 'enregistrerNote'],
10 ];
11
12 $uri = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH);
13 $uri = ($uri !== '/' && str_ends_with($uri, '/')) ? rtrim($uri, '/') : $uri;
14 $routeFound = false;
15
16 foreach ($Routes as $route) {
17     if ($route[0] === $uri) {
18         $controllerFile = dirname(__DIR__) . '/controllers/' . $route[1];
19         $action = $route[2];
20         if (file_exists($controllerFile)) {
21             require_once $controllerFile;
22             if (function_exists($action)) {
23                 $action();
24                 $routeFound = true;
25                 break;
26             }
27         }
28     }
29 }
30 if (!$routeFound) {
31     http_response_code(404);
32     echo "<h1>404 - Page non trouvee</h1>";
33 }

```

Listing 4.2 – Système de routage procédural sur la branche main

4.2.3 3. Le Contrôleur et la Logique de Saisie Procéduraux

Dans les contrôleurs procéduraux (AuthController.php, NoteController.php), la gestion de la session, la récupération de la base de données et l'inclusion de la vue étaient complètement enchevêtrées :

```

1  <?php
2  // app/controllers/AuthController.php (Branche main)
3  require_once dirname(__DIR__) . '/models/UtilisateurModel.php';
4
5  function login(): void {
6      init_session();
7      if (is_logged_in() && $_SERVER['REQUEST_METHOD'] === 'GET') {
8          header('Location: /gestion');
9          exit;
10     }
11     $error = null;
12     if ($_SERVER['REQUEST_METHOD'] === 'POST') {
13         $email = $_POST['email'] ?? '';
14         $password = $_POST['password'] ?? '';
15         if (!empty($email) && !empty($password)) {
16             $pdo = connexionDB(); // Instanciation explicite
17             $user = verify_user_credentials($pdo, $email, $password); //

```

Passage de \$pdo

```
18         if ($user) {
19             set_session('user', $user);
20             header('Location: /gestion');
21             exit;
22         } else {
23             $error = "Email ou mot de passe incorrect.";
24         }
25     }
26 }
27 require_once dirname(__DIR__) . '/views/PageConnexion.html.php';
28 }
```

Listing 4.3 – Contrôleur d'authentification procédural

4.3 Bilan de la Dette Technique Procédurale

Points Noirs de la Version Procédurale

- ✗ **Propagation Forcée de la Ressource PDO :** L'obligation de transmettre `$pdo` à travers chaque couche applicative polluait les signatures fonctionnelles et créait un couplage fort avec le pilote SQL.
- ✗ **Absence d'Encapsulation des Vues :** Les vues accédaient directement aux variables définies dans la fonction appelante sans isolation de portée.
- ✗ **Inflexibilité du Routage :** Impossibilité de mapper une même URL à deux contrôleurs différents selon qu'il s'agisse d'un GET (affichage) ou d'un POST (traitement).
- ✗ **Difficulté Majeure de Testabilité :** Impossible de créer des doubles de test (*mocks*) ou d'exécuter des tests automatisés indépendamment d'une véritable base PostgreSQL active.

Démarche Méthodologique & Stratégie de Migration

5.1 Principes Directeurs de la Refactorisation

La refactorisation d'une application existante sans interrompre la logique métier ni altérer les données sous-jacentes impose une méthodologie rigoureuse. Selon la définition de Martin Fowler, le refactoring consiste à « *modifier la structure interne d'un logiciel pour le rendre plus compréhensible et moins coûteux à modifier, sans altérer son comportement observable* ».

La migration de **NotesEcole** a suivi une démarche ascendante (*Bottom-Up*), partant des couches d'infrastructure de bas niveau jusqu'aux couches de présentation et de routage.

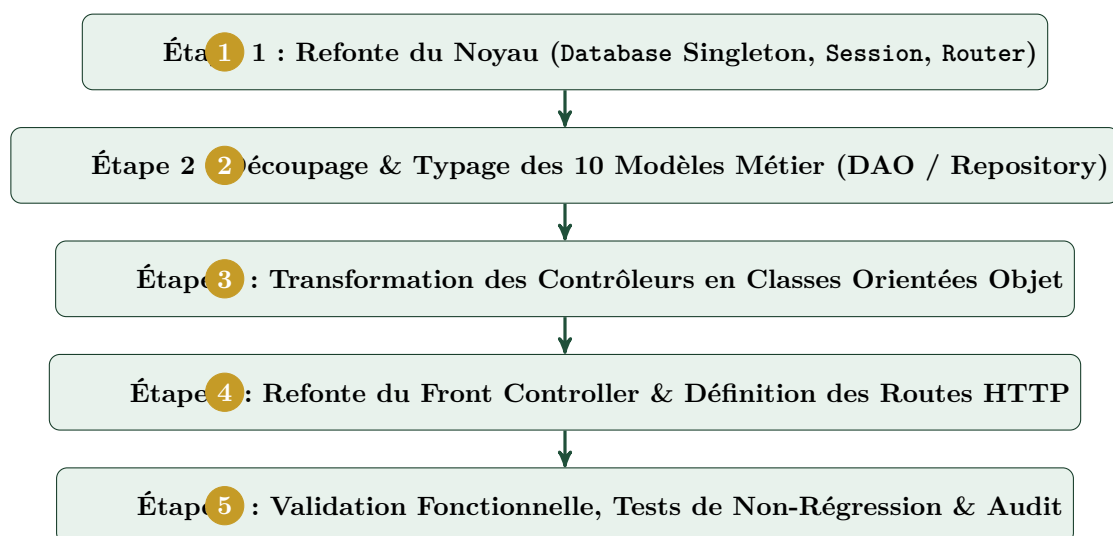


FIGURE 5.1 – Chronologie séquentielle des phases de refactorisation

5.2 Détail des Étapes de Transition

5.2.1 Étape 1 : Sécurisation et Unification du Noyau (app/core/)

La première priorité a consisté à éliminer la variable globale `$pdo` et les fonctions éparses.

- Conception de la classe `Database` appliquant le patron de conception **Singleton**. La connexion PostgreSQL est établie de manière paresseuse (*lazy-loading*) à la première invocation de `Database::getInstance()`.
- Implémentation de la classe utilitaire statique `Session` encapsulant `$_SESSION`, avec gestion des messages flash transitoires et contrôle des accès (`requireLogin()`).
- Création de la classe `Router` permettant l'enregistrement fluide des routes selon le verbe HTTP (`$router->get()`, `$router->post()`) et le dispatching dynamique.

5.2.2 Étape 2 : Éclatement et Spécialisation des Modèles (app/models/)

Dans la version procédurale, certaines tables n'avaient pas de modèle propre (ex. `inscriptions`, `eleves`, `roles`, `matiere_classes` étaient traitées au sein de requêtes ad-hoc dans `NoteModel.php`). La refonte a créé une classe dédiée pour chacune des 10 tables du schéma : `AnneeScolaireModel`, `ClasseModel`, `EleveModel`, `InscriptionModel`, `RoleModel`, `UtilisateurModel`, `MatiereModel`, `MatiereClasseModel`, `PeriodeModel`, `EvaluationModel`.

5.2.3 Étape 3 : Refactorisation des Contrôleurs (app/controllers/)

Les fonctions procédurales ont été converties en méthodes au sein des classes `AuthController` et `NoteController`.

- Injection des modèles nécessaires dès le constructeur `__construct()`.
- Encapsulation des méthodes utilitaires communes (`render()` pour l'extraction de données vers les vues, `redirect()` pour les en-têtes HTTP de redirection).
- Maintien de l'état des sélections utilisateurs synchronisé entre `$_GET` et la session.

5.2.4 Étape 4 : Modernisation du Point d'Entrée Unique (public/index.php)

Le script d'entrée `public/index.php` est devenu un **Front Controller** pur et déclaratif, initialisant le système, déclarant les routes et démarrant le routeur :

```

1 <?php
2 // public/index.php (Branche POO)
3
4 require_once dirname(__DIR__) . '/app/core/Database.php';
5 require_once dirname(__DIR__) . '/app/core/Session.php';
6 require_once dirname(__DIR__) . '/app/core/Router.php';
7
8 // Inclusions des 10 modeles et des controleurs
9 // ...
10
```

```
11 Session::start();
12
13 $router = new Router();
14 $router->get('/', 'AuthController', 'login');
15 $router->get('/login', 'AuthController', 'login');
16 $router->post('/login', 'AuthController', 'login');
17 $router->get('/logout', 'AuthController', 'logout');
18 $router->get('/gestion', 'NoteController', 'index');
19 $router->post('/gestion/save', 'NoteController', 'save');
20
21 $router->dispatch();
```

Listing 5.1 – Front Controller déclaratif en POO

5.3 Stratégie de Gestion de Version avec Git

Pour préserver l'historique et assurer une traçabilité intégrale :

- La branche **main** a été conservée avec son historique jusqu'au tag **v1.1.0**, témoignant de l'état fonctionnel procédural.
- Une branche dédiée **P00** a été créée pour accueillir la restructuration intégrale du code.
- Un commit atomique majeur a été enregistré : **0f6083d Refactorisation complete du projet en POO & MVC sur la branche P00**, modifiant 20 fichiers (+853 insertions, -489 suppressions).

Conception de la Nouvelle Architecture POO & MVC

6.1 Le Patron MVC Appliqué à NotesEcole

L'architecture logicielle cible adoptée sur la branche P00 repose sur le patron fondamental **Modèle-Vue-Contrôleur (MVC)**, garantissant une séparation stricte des préoccupations :

- **Le Modèle (Model)** : Représente la couche d'accès aux données et d'application des règles métier. Il communique avec la base de données PostgreSQL via la couche d'abstraction Database et renvoie des données structurées.
- **La Vue (View)** : Représente l'interface utilisateur graphique (fichiers `.html.php`). Elle n'effectue aucun calcul métier ni requête SQL, se contentant d'afficher les données qui lui sont transmises par le contrôleur.
- **Le Contrôleur (Controller)** : Fait le lien entre la requête de l'utilisateur (HTTP), les modèles et la vue. Il réceptionne les requêtes dispatchées par le **Router**, interroge les modèles, applique les filtres, gère les sessions et choisit la vue à afficher.

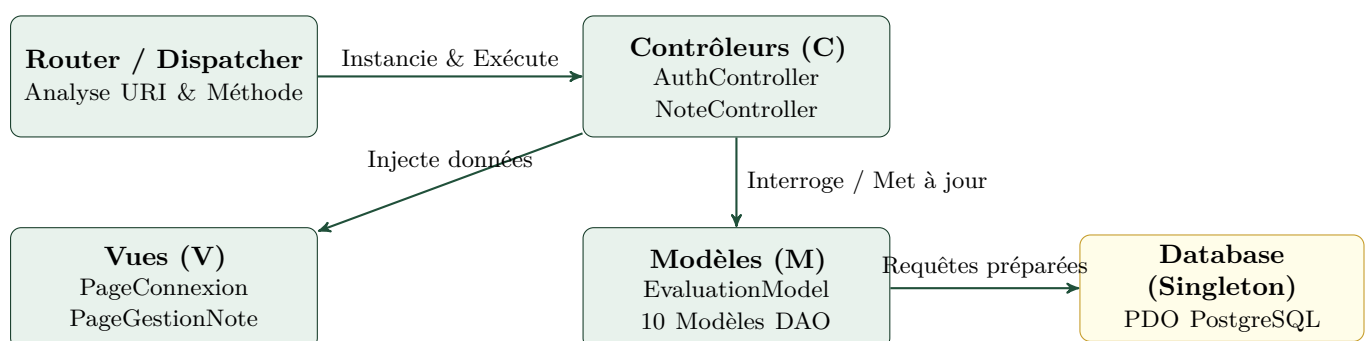


FIGURE 6.1 – Flux d'interactions de l'architecture MVC cible de NotesEcole

6.2 Le Noyau Applicatif (app/core/)

6.2.1 1. Le Patron Singleton pour la Classe Database

Afin d'éviter l'ouverture anarchique de multiples connexions TCP vers PostgreSQL (ce qui saturerait rapidement le pool de connexions du serveur), la classe `Database` applique le patron de conception **Singleton**.

Le Patron Singleton

Le Singleton garantit qu'une classe ne possède qu'une **seule et unique instance** durant tout le cycle de vie de la requête HTTP, et fournit un point d'accès global à cette instance via une méthode statique.

```
1 <?php
2 // app/core/Database.php (Extrait)
3
4 class Database {
5     private static ?Database $instance = null;
6     private ?PDO $pdo = null;
7
8     // Constructeur prive : interdit l'instanciation directe via new
9     Database()
10
11     private function __construct() {
12         try {
13             $this->pdo = new PDO(
14                 "pgsql:host=localhost;dbname=notes_ecole;port=5433",
15                 "ichigo",
16                 "password"
17             );
18             $this->pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
19                 PDO::FETCH_ASSOC);
20             $this->pdo->setAttribute(PDO::ATTR_ERRMODE,
21                 PDO::ERRMODE_EXCEPTION);
22         } catch (Exception $ex) {
23             die('Erreur de connexion a la base de donnees : ' .
24                 $ex->getMessage());
25         }
26     }
27
28     // Point d'accès unique a l'instance
29     public static function getInstance(): Database {
30         if (self::$instance === null) {
31             self::$instance = new self();
32         }
33         return self::$instance;
34     }
35
36     public function getConnection(): PDO {
```

```

32     return $this->pdo;
33 }
34
35 public function executeQuery(string $sql, array $datas = [], bool
    $single = false): array {
36     $statement = $this->prepare($sql, $datas);
37     $result = $single ? $statement->fetch() : $statement->fetchAll();
38     return $result !== false ? $result : [];
39 }
40
41 public function executeUpdate(string $sql, array $datas = []): int {
42     $statement = $this->prepare($sql, $datas);
43     if (str_starts_with(strtoupper(trim($sql)), 'INSERT')) {
44         return (int) $this->pdo->lastInsertId();
45     }
46     return $statement->rowCount();
47 }
48
49 // Gestion transactionnelle
50 public function beginTransaction(): bool { return
    $this->pdo->beginTransaction(); }
51 public function commit(): bool { return $this->pdo->commit(); }
52 public function rollBack(): bool { return $this->pdo->rollBack(); }
53 public function inTransaction(): bool { return
    $this->pdo->inTransaction(); }
54 }

```

Listing 6.1 – Implémentation complète du Singleton Database

6.2.2 2. Le Routeur Dynamique Orienté Verbes HTTP (Router.php)

La classe `Router` isole le tableau des routes enregistrées en fonction du verbe HTTP de la requête (GET ou POST) et résout dynamiquement la classe du contrôleur et l'action à appeler par réflexion et instanciation dynamique :

```

1  <?php
2  // app/core/Router.php
3
4  class Router {
5      private array $routes = [];
6
7      public function get(string $path, string $controller, string $action):
          void {
8          $this->routes['GET'][$path] = ['controller' => $controller, 'action'
              => $action];
9      }
10
11     public function post(string $path, string $controller, string $action):
        void {
12         $this->routes['POST'][$path] = ['controller' => $controller,
            'action' => $action];
13     }

```



```

14
15     public function dispatch(): void {
16         $uri = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH);
17         if ($uri !== '/' && str_ends_with($uri, '/')) {
18             $uri = rtrim($uri, '/');
19         }
20         $method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
21
22         if (isset($this->routes[$method][$uri])) {
23             $target = $this->routes[$method][$uri];
24             $controllerClass = $target['controller'];
25             $action = $target['action'];
26
27             if (class_exists($controllerClass)) {
28                 $controller = new $controllerClass();
29                 if (method_exists($controller, $action)) {
30                     $controller->$action();
31                     return;
32                 }
33             }
34         }
35
36         http_response_code(404);
37         $this->render404();
38     }
39 }

```

Listing 6.2 – La classe Router et son algorithme de dispatching

6.2.3 3. Gestionnaire de Session & Notifications Flash (Session.php)

La classe `Session` encapsule l'ensemble des opérations liées à l'état de l'utilisateur en mémoire :

```

1  <?php
2  // app/core/Session.php
3
4  class Session {
5      public static function start(): void {
6          if (session_status() === PHP_SESSION_NONE && !headers_sent()) {
7              session_start();
8          }
9      }
10
11     public static function set(string $key, mixed $value): void {
12         self::start();
13         $_SESSION[$key] = $value;
14     }
15
16     public static function get(string $key, mixed $default = null): mixed {
17         self::start();
18         return $_SESSION[$key] ?? $default;
19     }
20 }

```

```
21     public static function isLoggedIn(): bool {
22         self::start();
23         return self::get('user') !== null;
24     }
25
26     public static function requireLogin(): void {
27         if (!self::isLoggedIn()) {
28             header('Location: /login');
29             exit;
30         }
31     }
32
33     public static function setFlash(string $type, string $message): void {
34         self::set('flash_message', ['type' => $type, 'text' => $message]);
35     }
36
37     public static function getFlash(): ?array {
38         $flash = self::get('flash_message');
39         self::remove('flash_message');
40         return $flash;
41     }
42 }
```

Listing 6.3 – Gestion des sessions statiques et messages Flash

Étude Comparative Détaillée des Composants Métier

7.1 La Couche Modèle : Spécialisation et Encapsulation

Dans la version procédurale initiale, l'accès aux données était regroupé de manière désordonnée au sein de fichiers de fonctions tels que `NoteModel.php` (205 lignes) et `UtilisateurModel.php`. La refonte a découpé ces responsabilités en 10 classes modèles autonomes.

7.1.1 1. Le Modèle d'Évaluation (`EvaluationModel.php`)

La classe `EvaluationModel` constitue le cœur du domaine fonctionnel de gestion des notes. Elle centralise les algorithmes d'agrégation et les opérations d'écriture en base.

```
1  <?php
2  // app/models/EvaluationModel.php (Extrait)
3
4  class EvaluationModel {
5      private Database $db;
6      private PDO $pdo;
7
8      public function __construct() {
9          $this->db = Database::getInstance();
10         $this->pdo = $this->db->getConnection();
11     }
12
13     public function getElevesNotes(int $anneeId, int $classeId, int
14         $matiereId, int $periodeId): array {
15         $sql = "SELECT i.id AS inscription_id, e.id AS eleve_id, e.nom,
16             e.prenom, e.matricule,
17             ev.id AS evaluation_id,
```

```

16         COALESCE(ev.devoir1, 0) AS devoir1,
17         COALESCE(ev.devoir2, 0) AS devoir2,
18         COALESCE(ev.composition, 0) AS composition
19     FROM eleves e
20     JOIN inscriptions i ON i.eleve_id = e.id AND i.annee_id =
        :annee_id AND i.classe_id = :classe_id
21     LEFT JOIN evaluations ev ON ev.inscription_id = i.id
22         AND ev.matiere_id = :matiere_id
23         AND ev.periode_id = :periode_id
24     ORDER BY e.nom ASC, e.prenom ASC";
25
26     return $this->db->executeQuery($sql, [
27         'annee_id' => $anneeId,
28         'classe_id' => $classeId,
29         'matiere_id' => $matiereId,
30         'periode_id' => $periodeId
31     ]);
32 }
33
34 public static function calculerMoyenneEleve(mixed $d1, mixed $d2, mixed
    $comp): float {
35     $v1 = ($d1 !== null && $d1 !== '') ? (float)$d1 : 0.0;
36     $v2 = ($d2 !== null && $d2 !== '') ? (float)$d2 : 0.0;
37     $vc = ($comp !== null && $comp !== '') ? (float)$comp : 0.0;
38     return round(($v1 + $v2 + 2.0 * $vc) / 4.0, 2);
39 }
40
41 public static function getAppreciationNote(?float $moyenne): array {
42     if ($moyenne === null) return ['label' => 'Insuffisant', 'cls' =>
        'low'];
43     if ($moyenne >= 16) return ['label' => 'Tres bien', 'cls' => ''];
44     if ($moyenne >= 14) return ['label' => 'Bien', 'cls' => ''];
45     if ($moyenne >= 12) return ['label' => 'Assez bien', 'cls' => ''];
46     if ($moyenne >= 10) return ['label' => 'Passable', 'cls' => 'mid'];
47     return ['label' => 'Insuffisant', 'cls' => 'low'];
48 }
49 }

```

Listing 7.1 – Structure et méthodes de la classe EvaluationModel

7.1.2 2. Le Modèle d'Authentification (UtilisateurModel.php)

Dans la version POO, UtilisateurModel effectue la vérification des identifiants avec jointure sur la table roles, isolant totalement le mot de passe après contrôle :

```

1 public function verifyCredentials(string $email, string $password): ?array {
2     $sql = "SELECT u.id, u.nom, u.prenom, u.telephone, u.email, u.password,
3             r.id AS role_id, r.nomRole AS role_nom
4     FROM utilisateurs u
5     JOIN roles r ON r.id = u.role_id
6     WHERE u.email = :email";
7

```

```

8      $user = $this->db->executeQuery($sql, ['email' => $email], true);
9      if ($user && $user['password'] === $password) {
10         unset($user['password']); // Suppression du mot de passe de la
            mémoire
11         return $user;
12     }
13     return null;
14 }

```

Listing 7.2 – Vérification des identifiants au sein d'UtilisateurModel

7.2 La Couche Contrôleur : Orchestration et Injection de Dépendances

7.2.1 1. Le Contrôleur de Notes (NoteController.php)

`NoteController` illustre parfaitement la puissance de la POO en injectant au sein de son constructeur les 5 modèles métier nécessaires à son exécution :

```

1  <?php
2  // app/controllers/NoteController.php
3
4  class NoteController {
5      private AnneeScolaireModel $anneeModel;
6      private ClasseModel $classeModel;
7      private MatiereClasseModel $matiereClasseModel;
8      private PeriodeModel $periodeModel;
9      private EvaluationModel $evaluationModel;
10
11     public function __construct() {
12         $this->anneeModel = new AnneeScolaireModel();
13         $this->classeModel = new ClasseModel();
14         $this->matiereClasseModel = new MatiereClasseModel();
15         $this->periodeModel = new PeriodeModel();
16         $this->evaluationModel = new EvaluationModel();
17     }
18 }

```

Listing 7.3 – Constructeur et injection de dépendances dans NoteController

7.2.2 2. Sauvegarde par Lot et Robustesse de l'Action `save()`

L'action `save()` réceptionne la matrice de notes soumise via le formulaire HTML, assainit les entrées numériques et déclenche la persistance transactionnelle avant d'émettre un message Flash :

```

1  public function save(): void {
2      Session::requireLogin();
3

```

```

4      if (($_SERVER['REQUEST_METHOD'] ?? 'GET') === 'POST') {
5          $classeId = (int)($_POST['classe_id'] ??
              Session::get('selected_classe_id', 1));
6          $matiereId = (int)($_POST['matiere_id'] ??
              Session::get('selected_matiere_id', 1));
7          $periodeId = (int)($_POST['periode_id'] ??
              Session::get('selected_periode_id', 1));
8
9          $rawNotes = $_POST['notes'] ?? [];
10         $notesToSave = [];
11
12         foreach ($rawNotes as $inscriptionId => $n) {
13             $notesToSave[] = [
14                 'inscription_id' => (int)$inscriptionId,
15                 'devoir1' => isset($n['devoir1']) &&
                     trim((string)$n['devoir1']) !== '' ? (float)$n['devoir1']
                     : 0.0,
16                 'devoir2' => isset($n['devoir2']) &&
                     trim((string)$n['devoir2']) !== '' ? (float)$n['devoir2']
                     : 0.0,
17                 'composition' => isset($n['composition']) &&
                     trim((string)$n['composition']) !== '' ?
                     (float)$n['composition'] : 0.0
18             ];
19         }
20
21         $success = $this->evaluationModel->saveNotes($matiereId, $periodeId,
            $notesToSave);
22         if ($success) {
23             Session::setFlash('success', 'Les notes ont ete enregistrees
                avec succes en base de donnees.');
```

Listing 7.4 – Traitement de sauvegarde groupée et redirection PRG

Sécurité, Transactions ACID & Qualité du Code

8.1 Garantie d'Intégrité par les Transactions ACID

Dans un contexte scolaire, la saisie des notes s'effectue couramment par classe entière (soit 30 à 50 élèves par formulaire). L'écriture des notes en base implique autant d'opérations d'insertion ou de mise à jour que d'élèves inscrits.

8.1.1 Le Problème de l'Écriture Partielle

Sans gestion transactionnelle, si une coupure réseau ou une contrainte de validation échoue au 35^{ème} enregistrement, les 34 premiers élèves sont modifiés en base tandis que les 15 suivants demeurent inchangés. La base de données se retrouve alors dans un **état incohérent et partiel**.

Les Propriétés ACID

- **A – Atomicité** : La suite d'opérations est exécutée en totalité ou rejetée en totalité (*Tout ou Rien*).
- **C – Consistance** : Les contraintes d'intégrité de la base sont toujours respectées.
- **I – Isolation** : Les transactions concurrentes n'interfèrent pas entre elles.
- **D – Durabilité** : Une fois validées (*commit*), les modifications sont persistées durablement.

8.1.2 Implémentation Transactionnelle dans `saveNotes()`

La méthode `saveNotes()` de la classe `EvaluationModel` encapsule l'intégralité du traitement dans un bloc `try / catch` transactionnel :

```
1 public function saveNotes(int $matiereId, int $periodeId, array $notes):  
    bool {  
2     try {  
3         $this->db->beginTransaction(); // Demarrage de la transaction  
4  
5         $sqlCheck = "SELECT id FROM evaluations  
6                     WHERE inscription_id = :inscription_id  
7                     AND matiere_id = :matiere_id  
8                     AND periode_id = :periode_id";  
9         $sqlInsert = "INSERT INTO evaluations (inscription_id, matiere_id,  
10                    periode_id, devoir1, devoir2, composition)  
11                    VALUES (:inscription_id, :matiere_id, :periode_id,  
12                           :devoir1, :devoir2, :composition)";  
13  
14         $sqlUpdate = "UPDATE evaluations  
15                     SET devoir1 = :devoir1, devoir2 = :devoir2,  
16                           composition = :composition  
17                     WHERE id = :id";  
18  
19         foreach ($notes as $note) {  
20             $inscriptionId = (int) $note['inscription_id'];  
21             $d1 = (float) $note['devoir1'];  
22             $d2 = (float) $note['devoir2'];  
23             $comp = (float) $note['composition'];  
24  
25             $existing = $this->db->executeQuery($sqlCheck, [  
26                 'inscription_id' => $inscriptionId,  
27                 'matiere_id' => $matiereId,  
28                 'periode_id' => $periodeId  
29             ], true);  
30  
31             if (!empty($existing) && isset($existing['id'])) {  
32                 $this->db->executeUpdate($sqlUpdate, [  
33                     'id' => $existing['id'],  
34                     'devoir1' => $d1,  
35                     'devoir2' => $d2,  
36                     'composition' => $comp  
37                 ]);  
38             } else {  
39                 $this->db->executeUpdate($sqlInsert, [  
40                     'inscription_id' => $inscriptionId,  
41                     'matiere_id' => $matiereId,  
42                     'periode_id' => $periodeId,  
43                     'devoir1' => $d1,  
44                     'devoir2' => $d2,  
45                     'composition' => $comp  
46                 ]);  
47             }  
48         }  
49  
50         $this->db->commit(); // Validation globale atomique  
51         return true;  
52     } catch (Exception $e) {
```



```

49         if ($this->db->inTransaction()) {
50             $this->db->rollBack(); // Annulation immédiate en cas d'erreur
51         }
52         return false;
53     }
54 }

```

Listing 8.1 – Implémentation de l’atomicité transactionnelle dans EvaluationModel

8.2 Sécurisation contre les Injections SQL via Requêtes Préparées

Toutes les requêtes SQL du projet sans exception sont exécutées au moyen de **requêtes préparées PDO** avec liaison de paramètres nommés.

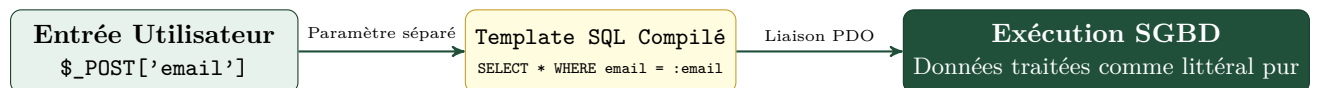


FIGURE 8.1 – Mécanisme d’immunisation contre les injections SQL par séparation du code et des données

8.3 Typage Strict et Modernité PHP 8.x

La refonte en POO tire pleinement parti des fonctionnalités introduites dans les versions récentes de PHP (8.0, 8.1, 8.2) :

- **Propriétés Typées dans les Classes** : Déclaration explicite des types d’attributs (`private Database $db;`, `private ?PDO $pdo = null;`), interdisant toute assignation de type incohérent au moment de l’exécution.
- **Types d’Union et Types Nullable** : Emploi de `string|false`, `?array`, `?int` permettant une signature de fonctions transparente et rigoureuse.
- **Fonctions Natives Modernes** : Utilisation de `str_starts_with()` et `str_ends_with()` pour la détection sémantique des requêtes `INSERT` et le formatage des URIs dans le routeur.

Guide de Déploiement & Perspectives d'Évolution

9.1 Guide de Déploiement et Configuration de l'Application

Pour déployer l'application **NotesEcole** dans un environnement de test ou de production, les étapes suivantes sont préconisées.

9.1.1 Prérequis Système

- **PHP** : Version 8.2 ou supérieure avec les extensions activées : `pdo`, `pdo_pgsql`, `session`, `filter`, `json`.
- **Base de Données** : PostgreSQL 14+ (configuré sur le port 5433 ou standard 5432).
- **Serveur Web** : Nginx, Apache (avec module `mod_rewrite` actif) ou le serveur interne PHP pour le développement.

9.1.2 Initialisation de la Base de Données

L'initialisation du schéma relationnel et des données de démonstration s'effectue via le script `requete.sql` :

```
1 # Creation de la base de donnees et import des tables
2 psql -h localhost -p 5433 -U ichigo -d postgres -c "CREATE DATABASE
   notes_ecole;"
3 psql -h localhost -p 5433 -U ichigo -d notes_ecole -f requete.sql
```

Listing 9.1 – Commandes d'initialisation de la base PostgreSQL

9.1.3 Lancement de l'Application en Environnement Local

Pour démarrer le serveur de développement local avec pointage sur le dossier `public/` :

```
1 php -S localhost:8000 -t public
```

Listing 9.2 – Lancement du serveur PHP natif

L'application est immédiatement accessible à l'adresse <http://localhost:8000/login>.

9.2 Guide de Compilation du Document LaTeX

Le présent document a été conçu selon les standards académiques et peut être compilé via n'importe quelle distribution TeX moderne (**TeXLive 2022+**, **MacTeX**, **MikTeX**) ou importé directement sur la plateforme en ligne **Overleaf**.

9.2.1 Compilation en Ligne de Commande

Pour résoudre l'ensemble des références croisées, tables des matières, listes de figures et listings de code, une double passe de compilation est requise :

```
1 pdflatex -interaction=nonstopmode documentation_complete.tex
2 pdflatex -interaction=nonstopmode documentation_complete.tex
```

Listing 9.3 – Compilation de la documentation via pdflatex

9.3 Perspectives d'Évolution et Feuille de Route (*Roadmap*)

La refactorisation réussie vers la POO et le MVC pose des fondations saines permettant d'envisager des évolutions architecturales majeures :



FIGURE 9.1 – Feuille de route technique pour les versions futures de NotesEcole

- 1. Autoloading PSR-4 et Composer** : Remplacer les directives `require_once` manuelles par l'autoloading standardisé de Composer (`vendor/autoload.php`) sous l'espace de nommage `App\`.
- 2. Moteur de Rendu Dédié (Twig)** : Remplacer les inclusions de fichiers `.html.php` par des templates Twig afin d'offrir l'héritage de mise en page (*layout inheritance*) et l'échappement XSS automatique.
- 3. Suite de Tests Automatisés (PHPUnit)** : Écrire une suite de tests unitaires pour valider les formules de calcul de moyennes (`calculerMoyenneEleve`) et les règles d'arrondi.

4. **Interface API REST / Mobile :** Exposer les données d'évaluation via une API REST sécurisée par JWT, permettant la consultation des bulletins scolaires sur smartphone par les parents d'élèves.

Conclusion Générale & Bilan Technique

10.1 Synthèse de la Transformation Logicielle

La refactorisation du projet **NotesEcole** depuis une approche purement procédurale vers une architecture orientée objet structurée en **MVC (Modèle-Vue-Contrôleur)** marque une étape décisive dans la pérennisation et la professionnalisation de l'application.

En substituant des fonctions globales et des inclusions manuelles par des classes cohérentes dotées de responsabilités unitaires (*Single Responsibility Principle*), le système a éliminé sa dette technique originelle tout en renforçant son niveau de sécurité et d'évolutivité.

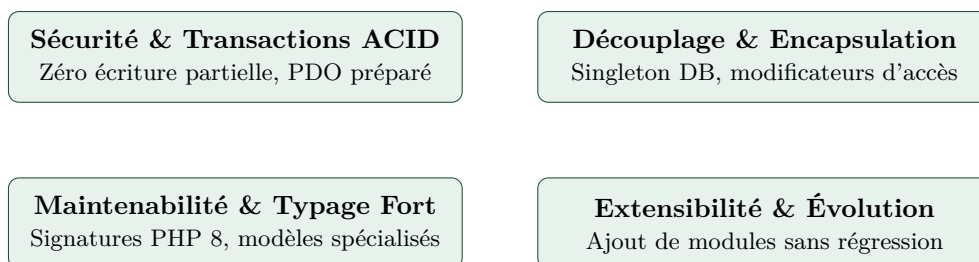


FIGURE 10.1 – Les quatre piliers de la valeur ajoutée apportée par la refonte POO

10.2 Bilan Quantitatif et Qualitatif

10.2.1 Indicateurs Qualitatifs

- **Élimination du Couplage Transversal** : La ressource de connexion PDO n'est plus véhiculée à travers des dizaines de fonctions intermédiaires. La classe `Database` gère son propre cycle de vie et protège la ressource.

- **Robustesse Métier** : Le calcul des moyennes scolaires, pondérations et arrondis à deux décimales (`ROUND(..., 2)`) est désormais encapsulé dans `EvaluationModel`, avec filtrage garanti des notes nulles.
- **Expérience Utilisateur Améliorée** : La persistance des filtres de sélection en session (`classe_id`, `matiere_id`, `periode_id`) évite les réinitialisations intempestives lors de la navigation ou de la sauvegarde des notes.

10.2.2 Indicateurs Métriques du Dépôt Git

La refonte concentrée sur la branche `P00` témoigne d'un remaniement en profondeur :

- **20 fichiers** impactés au cœur du projet.
- **+853 lignes de code** structurées, documentées et typées.
- **-489 lignes de code** procédural déprécié supprimées.
- **10 classes modèles** créées pour refléter l'intégralité du schéma relationnel PostgreSQL.

Dictionnaire de Données Exhaustif

A.1 Détail des Tables du Schéma PostgreSQL

Le tableau A.1 répertorie l'intégralité des colonnes, types et contraintes du schéma de la base de données `notes_ecole`.

Table	Champ	Type	Null / Def	Description & Contraintes
anneeScolaires	id	SERIAL	PK	Identifiant unique de l'année scolaire
	nom	VARCHAR(30)	NOT NULL	Libellé de l'année scolaire (UNIQUE, ex. 2025-2026)
	date actif	DATE INT	DEFAULT NOW NULL	Date d'ouverture de l'année 1 si active, 0 sinon
eleves	id	SERIAL	PK	Identifiant unique de l'élève
	nom	VARCHAR(50)	NOT NULL	Nom de famille de l'élève
	prenom	VARCHAR(50)	NOT NULL	Prénom de l'élève
	matricule	VARCHAR(30)	NOT NULL	Numéro matricule unique (UNIQUE, ex. JE-26002)
classes	id	SERIAL	PK	Identifiant de la classe
	nomClasse	VARCHAR(30)	NOT NULL	Nom de la cohorte (UNIQUE, ex. 3em A)
inscriptions	id	SERIAL	PK	Identifiant unique de l'inscription
	annee_id	INT	FK	Référence vers anneeScolaires(id) ON DELETE CASCADE
	eleve_id	INT	FK	Référence vers eleves(id) ON DELETE CASCADE
	classe_id	INT	FK	Référence vers classes(id) ON DELETE CASCADE
roles	id	SERIAL	PK	Identifiant du rôle
	nomRole	VARCHAR(50)	NOT NULL	Nom du rôle (Direction, Professeur, Surveillant)
utilisateurs	id	SERIAL	PK	Identifiant de l'utilisateur
	nom	VARCHAR(50)	NOT NULL	Nom de famille
	prenom	VARCHAR(50)	NOT NULL	Prénom
	telephone	VARCHAR(30)	NOT NULL	Numéro de téléphone (UNIQUE)
	email	VARCHAR(30)	NOT NULL	Adresse email de connexion (UNIQUE)
	password	VARCHAR	NULL	Mot de passe haché ou en clair
	role_id	INT	FK	Référence vers roles(id)
matieres	id	SERIAL	PK	Identifiant de la discipline
	nomMatiere	VARCHAR(40)	NOT NULL	Nom de la matière (UNIQUE, ex. Mathématique)
matiere_classes	id	SERIAL	PK	Identifiant de la liaison
	classe_id	INT	FK	Référence vers classes(id) ON DELETE CASCADE
	matiere_id	INT	FK	Référence vers matieres(id) ON DELETE CASCADE
periodes	id	SERIAL	PK	Identifiant du trimestre
	nomPeriode	VARCHAR(50)	NOT NULL	Libellé de la période (Trimestre 1, 2, 3)
evaluations	id	SERIAL	PK	Identifiant de la note
	inscription_id	INT	FK	Référence vers inscriptions(id) ON DELETE CASCADE
	matiere_id	INT	FK	Référence vers matieres(id) ON DELETE CASCADE
	periode_id	INT	FK	Référence vers periodes(id) ON DELETE CASCADE
	devoir1	NUMERIC(4,2)	NULL	Note de contrôle continu 1 (/20)
	devoir2	NUMERIC(4,2)	NULL	Note de contrôle continu 2 (/20)
	composition	NUMERIC(4,2)	NULL	Note d'épreuve terminale (/20, coef 2)

Diagramme de Classes UML Global

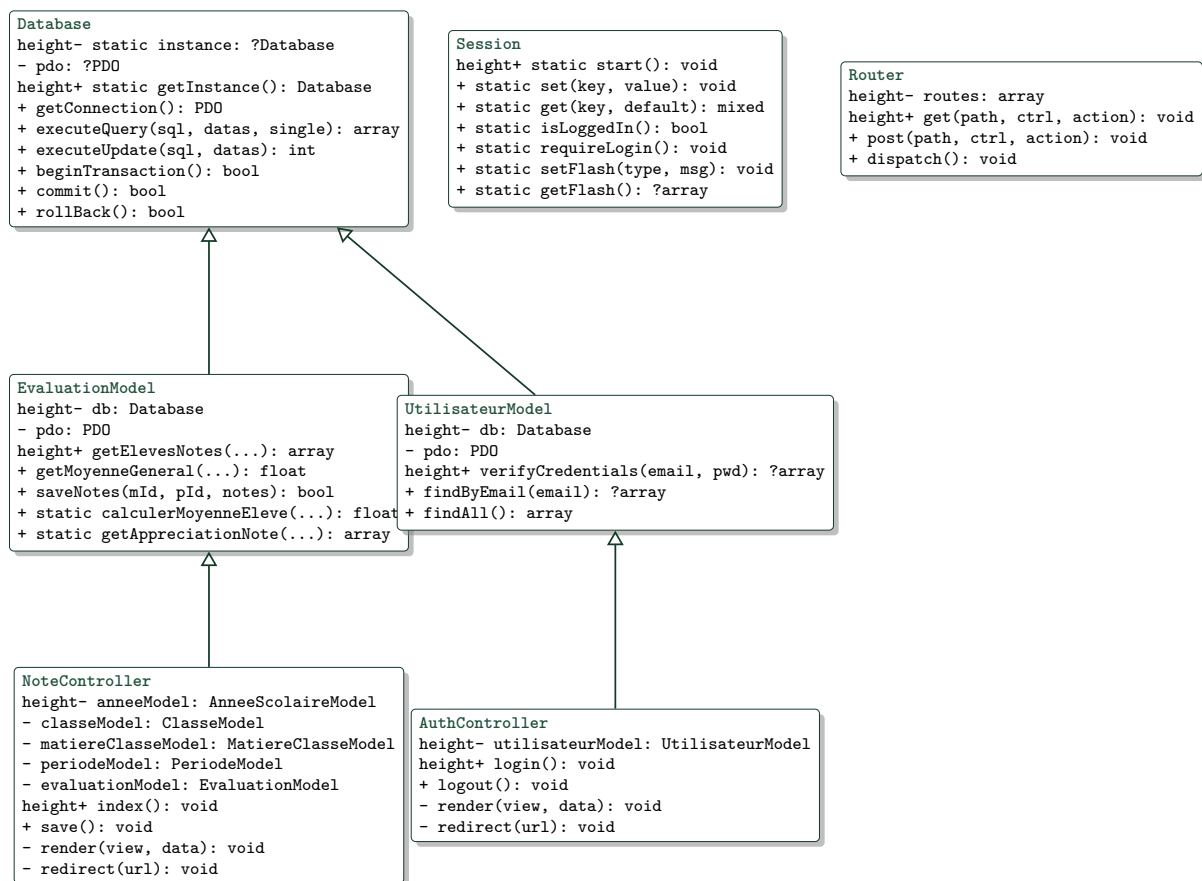


FIGURE B.1 – Diagramme de classes UML global de l'architecture POO & MVC

Glossaire des Termes d'Ingénierie Logicielle

ACID (Atomicité, Consistance, Isolation, Durabilité) : Ensemble de propriétés garantissant la fiabilité et la cohérence des transactions au sein d'une base de données relationnelle.

DAO (Data Access Object) : Patron de conception permettant de séparer la logique d'accès aux données des règles de traitement métier.

Encapsulation : Principe de programmation objet consistant à regrouper les données et les méthodes agissant sur ces données au sein d'une même entité, en masquant sa structure interne.

Front Controller : Patron d'architecture centralisant le traitement de toutes les requêtes entrantes d'une application Web en un point d'entrée unique (`public/index.php`).

MVC (Modèle-Vue-Contrôleur) : Patron d'architecture logicielle séparant l'application en trois composants interconnectés : la logique métier (Modèle), l'interface visuelle (Vue) et la gestion des flux (Contrôleur).

PRG (Post/Redirect/Get) : Patron de conception Web évitant la double soumission de formulaires en redirigeant l'utilisateur via une requête `GET` immédiatement après le traitement réussi d'un `POST`.

Singleton : Patron de conception restreignant l'instanciation d'une classe à une unique instance globale partagée.

Injection SQL : Vulnérabilité de sécurité permettant à un attaquant d'injecter des commandes SQL non autorisées au sein des requêtes d'une application.

Références & Bibliographie

1. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).** *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
2. **Fowler, M. (2018).** *Refactoring : Improving the Design of Existing Code* (2nd Edition). Addison-Wesley Professional.
3. **Martin, R. C. (2008).** *Clean Code : A Handbook of Agile Software Craftsmanship*. Prentice Hall.
4. **PHP Documentation Group (2026).** *PHP 8.2 PDO Reference Manual*. Disponible en ligne sur : <https://www.php.net/docs.php>.
5. **The PostgreSQL Global Development Group (2026).** *PostgreSQL 16 Documentation - Data Aggregation and Array Functions*. Disponible sur : <https://www.postgresql.org/docs/16/>.